HCLSoftware

Deploy HCL DX Compose in Minutes

November 12, 2026



HCLSoftware

DX Webinar

An ongoing series of product-focused, hands-on sessions highlighting features and functionality, with an emphasis on demos and visuals.

Watch for your invite to our next session!

HCL **Digital Experience**

HCLSoftware

Webinar

Deploy HCL DX Compose in Minutes:

Live Demo & Expert Insights!

November 12, 2026 | 5 PM CET



Andrés Gorostidi Pulgar
EuropeSIP DX Expert



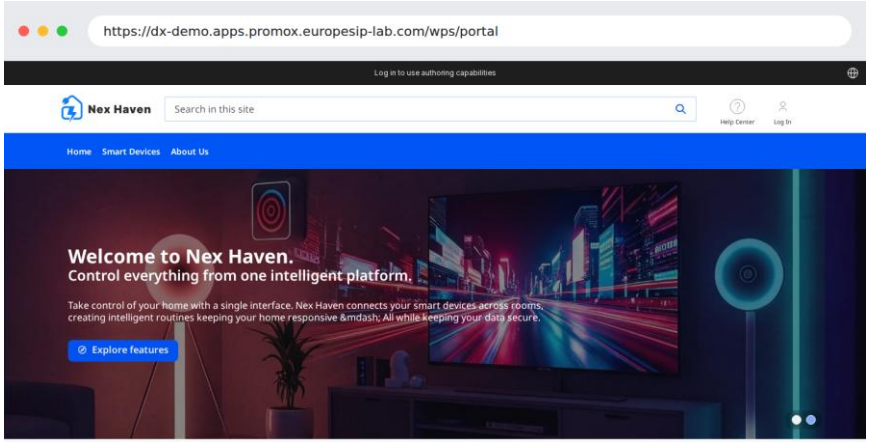
Herbert Hilhorst
HCLSoftware TX
Customer Success Engineer

Deploy HCL DX Compose in Minutes

Ready to take the guesswork out of deploying DX Compose on Kubernetes?

Hands-on session with our trusted Business Partner, EuropeSIP, where we strip away the complexity and build a deployment live—step by step, iteration by iteration.

If you already hold DX licenses, you can deploy DX Compose across your non-production environments today at no extra licensing hurdle!



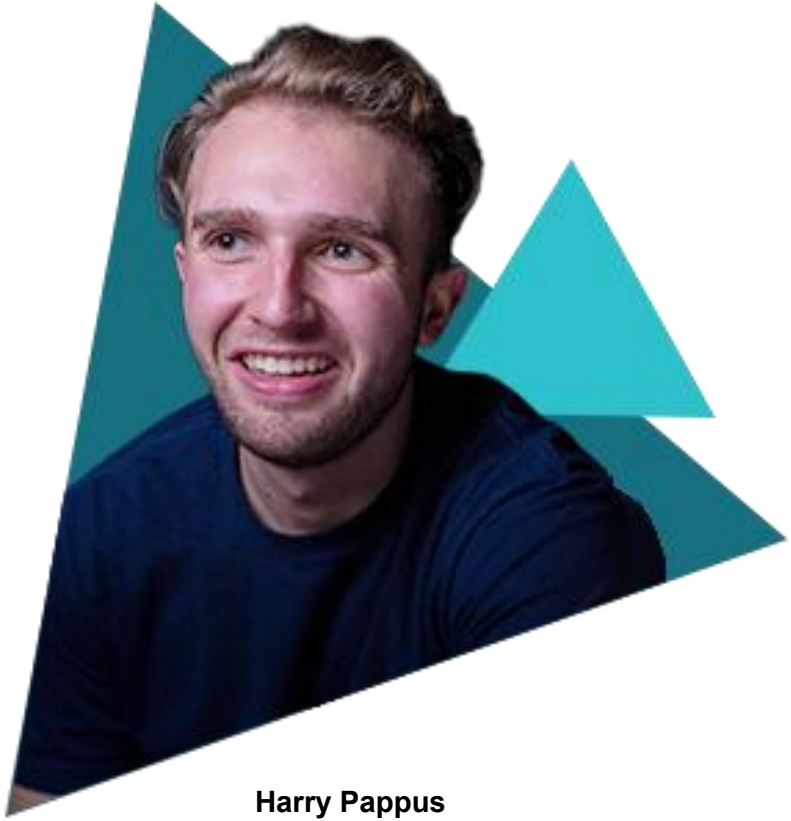
What we build today: HCL DX Compose with the Nex Haven demo site

europesip

Deploy DX Compose in Action

Scenario: Harry Pappus, a DX admin, deploys DX Compose, through a practical, step-by-step incremental approach—building real capabilities one iteration at a time:

1. Base DX with embedded Derby
2. LDAP via Kubernetes Secrets
3. External database
4. AI assistant (IQ)
5. Search with OpenSearch



Harry Pappus

DX Admin, Nex Haven

europesip

Our starting point: the cluster

Kubernetes

- **k3s v1.35.5**, CNCF-certified Kubernetes, managed with Rancher
- **3 control-plane + 3 worker** nodes, Ubuntu 24.04, x86-64
- Workstation: **kubectl** and **Helm 4.3**
- Access to the **HCL Harbor** (hclcr.io) with our CLI secret

DNS, ingress, storage

- **DNS** at our provider: *.apps.promox.europesip-lab.com already points to the cluster
- **Ingress:** ingress-nginx behind MetalLB
- **StorageClasses** (Longhorn 1.12): **dx-deploy-rwo** (ReadWriteOnce) and **dx-deploy-rwx** (ReadWriteMany, for DAM)

Minimum free resources

- **Base DX:** ~5 CPU, ~11 GiB
- **+ LDAP and PostgreSQL:** +0.5 CPU, +1 GiB
- **+ IQ and LiteLLM:** +0.5 CPU, +1.5 GiB
- Start-up peak with limits: ~8 CPU, ~14 GiB, spread over the nodes

The external dependencies

OpenLDAP · pre-installed

- Demo users (andres, macarena…) and groups (**dxadmins**, dxusers)
- openldap.dx-shared:636 (LDAPS)
- Files: **lab-ldap/**

PostgreSQL 18 · pre-installed

- CloudNativePG operator
- **dx_core** prepared for the Web Engine with the HCL table script
- dx-pg-rw.dx-shared:5432 (SSL only)
- Files: **lab-bbdd/**

LiteLLM · live in Lab 4

- Proxy between IQ and the AI models
- OpenRouter → **Claude Sonnet 4.6** / Haiku 4.5
- litellm.dx-shared:4000
- Files: **lab-iq/**

Third-party software, not HCL. OpenLDAP and PostgreSQL were installed before the session, with SSL, as a corporate directory and database would be. LiteLLM, the prerequisite of IQ, is installed live at the start of Lab 4, with IQ. · **dx-demo** = HCL software only · **dx-shared** = these third-party tools.

AGENDA

Today: a production-style deployment

Lab	What we build	⏱ Run time
0	**Prerequisites** Check the preparation · DX namespace · HCL charts	**~2 min**
1	**Base DX** DX Compose CF239 on Derby, published	**~6 min***
2	**LDAP** DX users from OpenLDAP, e-mail sign-in, LDAPS	**~12 min**
3	**External database** Web Engine data: Derby → PostgreSQL over SSL	**~13 min**
4	**AI assistant** LiteLLM, then HCL DX IQ (OpenRouter → Claude)	**~9 min**
5	**Search** OpenSearch and Search V2	**~7 min**
Total run time		**~49 min**

Looking ahead: towards a GitOps world

Declarative · GitOps-ready

- YAML files instead of imperative commands, Secrets included
- GitOps in large companies: Argo CD or Flux
- Secrets from a manager (HashiCorp Vault, OpenBao, Azure Key Vault) via External Secrets Operator
- Nothing readable in Git: Sealed Secrets or SOPS

SSL, as in a company

- LDAPS on port 636, not plain LDAP on 389
- PostgreSQL only over SSL (sslmode=verify-full)
- Server certificates signed by corporate CAs, trusted by Liberty

Helm 4

- Helm 4 instead of Helm 3
- Small flag differences: --wait=legacy, --force-conflicts
- With Helm 3: --wait, and no --force-conflicts

Good practices

- Pinned chart and image versions
- One values file per environment, in Git: it names the Secrets, never holds them
- cert-manager to renew the certificates

The HCL documentation shows imperative commands, like kubectl create secret. Here we create the same objects with kubectl apply and YAML files: same result, equally supported. Both ways are valid; we simply prefer YAML.

Prerequisites: DX namespace and HCL charts

⏱ ~2 min

What we do

- Check what is already in **dx-shared**: OpenLDAP and PostgreSQL
- Create the DX namespace, **dx-demo**: the generic YAML, or the OpenShift one with the annotations HCL requires
- Sign in to the **HCL Harbor** and pull the charts of DX, IQ and Search

Commands · steps 0-1 to 0-3

```
# 0-1 · check the environment
kubectl get nodes
kubectl -n dx-shared get pods             # OpenLDAP, PostgreSQL
kubectl -n dx-shared get clusters.postgresql.cnpg.io
# 0-2 · the DX namespace
cat lab-base/namespace-dx.yaml             # any Kubernetes
cat lab-base/namespace-dx.openshift.yaml   # OpenShift
kubectl apply -f lab-base/namespace-dx.yaml
# 0-3 · HCL Harbor: sign in and pull the charts
helm registry login hclcr.io -u <user> -p <cli-secret>
helm pull oci://hclcr.io/dx-compose/hcl-dx-deployment \
     --version 2.50.0 -d charts
helm pull oci://hclcr.io/dx/hcl-dx-iq --version 1.0.3 -d charts
helm pull oci://hclcr.io/dx-compose/hcl-dx-search \
     --version 2.37.0 -d charts
```

Key values

- **dx-shared**: third-party tools · **dx-demo**: HCL software only
- OpenShift: **openshift.io/sa.scc.*** annotations (restricted-v2)
- hclcr.io: your Harbor user and **CLI secret**
- Charts: **hcl-dx-deployment 2.50.0**, hcl-dx-iq 1.0.3, hcl-dx-search 2.37.0

▶ **Live demo:** show both namespace files and apply one, then sign in to the HCL Harbor and pull the three charts. The images are pulled later by the cluster.

📖 HCL documentation (CF239): **DX Compose · Preparation**

Base DX: DX Compose on Derby

⏱ ~6 min*

What we do

- Secrets: pull secret for hclcr.io, **wpsadmin** and a TLS certificate for HAProxy
- Chart **hcl-dx-deployment 2.50.0** (CF239): Web Engine, HAProxy, DAM, Persistence Node
- Check it with kubectl port-forward, then publish it: an Ingress that re-encrypts towards HAProxy
- Result: DX Compose on **Derby**, basic user registry

Commands · steps 1-1 to 1-5

```
# 1-1 · secrets regcred + web-engine-secret, TLS for HAProxy
kubectl apply -f lab-base/secrets.yaml
openssl req -x509 -newkey rsa:2048 -nodes ... -out lab-base/dx-tls.crt
kubectl -n dx-demo create secret tls dx-tls-cert --cert=... --key=...
# 1-2 · install DX Compose
helm upgrade --install dx-deployment \
     charts/hcl-dx-deployment-2.50.0.tgz -n dx-demo \
     -f lab-base/custom-values.yaml --timeout 30m --wait=legacy
# 1-3 · check DX with kubectl port-forward, before the Ingress
kubectl -n dx-demo port-forward svc/dx-deployment-haproxy 8443:443 &
curl -sk -o /dev/null -w '%{http_code}' https://localhost:8443/wps/portal
# 1-4 · publish it
kubectl apply -f lab-base/ingress-dx.yaml
# 1-5 · check the portal (optional)
```

Key values

- applications: **webEngine** true, core false, openLdap false
- networking.webEngine.host: **dx-demo.apps.promox.europesip-lab.com**
- **dx-deploy-rwo**: Web Engine, Persistence Node, License Manager
- **dx-deploy-rwx**: DAM binaries
- images: hclcr.io + **regcred**

▶ **Live demo:** walk through **lab-base/custom-values.yaml**, then helm install, watch the pods start and log in to /wps/portal as wpsadmin.

📖 **HCL documentation (CF239):** **DX Compose · Installation**

LDAP: DX users from OpenLDAP

⏱ ~12 min

What we do

- DX authenticates against the pre-installed OpenLDAP: the **native LDAP block** of the chart, bind credentials in a Secret
- Administrator role for the **dxadmins** group (XMLAccess)
- Advanced: **configuration overrides** for sign-in by e-mail and the preferred language
- **LDAPS** with the corporate CAs in the Liberty truststore; then plain LDAP is closed

Commands · steps 2-1 to 2-4

```
# 2-1 · the bind secret, then the basic LDAP layer
kubectl apply -f lab-ldap/secrets.yaml
helm upgrade --install dx-deployment ... -f lab-base/custom-values.yaml \
     -f lab-ldap/custom-values.ldap-basic.yaml
# 2-2 · Administrator role for dxadmins (XMLAccess)
curl -sk -u wpsadmin:<password> --data-binary @lab-ldap/admin-role.xml \
     https://<host>/wps/config
# 2-3 · the corporate CAs, then the complete layer (overrides, LDAPS)
kubectl -n dx-demo create secret generic dx-corporate-cas \
     --from-file=internals-ca/corporate-directory-ca.crt \
     --from-file=internals-ca/corporate-database-ca.crt
helm upgrade --install dx-deployment ... -f lab-base/custom-values.yaml \
     -f lab-ldap/custom-values.ldap.yaml
# 2-4 · only LDAPS: OpenLDAP requires TLS (olcSecurity: tls=1)
```

Key values

- ldap.type: **other** · port 389, then **636** and **sslEnabled: true**
- customLdapSecret: **dx-ldap-bind**
- configOverrideFiles: **loginProperty mail**, **wplc-add-property**
- customTruststoreSecrets: **dx-corporate-cas**
- groups: groupOfUniqueNames / uniqueMember

▶ **Live demo:** the basic LDAP layer, the Administrator role and andres; macarena's e-mail fails until the overrides and LDAPS (2-3); then plain LDAP is closed (2-4).

📖 **HCL documentation (CF239):** **DX Compose · LDAP configuration**

External database: Derby → PostgreSQL

⏱ ~13 min

What we do

- The **WebEngine** transfers its data from Derby to **our existing PostgreSQL 18**
- Two secrets with the dbtype / dbdomain properties; JDBC over SSL
- One **helm upgrade** with useExternalDatabase: the chart **transfers** the data
- PostgreSQL needs the table script from HCL (early release): **lab-bbdd/setup-postgres-dx.sql**

Commands · steps 3-1 to 3-3

```
# 3-1 · secrets with the Web Engine database properties
kubectl -n dx-demo create secret generic \
     custom-credentials-webengine-dbtype-secret \
     --from-file=lab-bbdd/dx_dbtype.properties
kubectl -n dx-demo create secret generic \
     custom-credentials-webengine-dbdomain-secret \
     --from-file=lab-bbdd/dx_dbdomain.properties
# 3-2 · transfer Derby -> PostgreSQL
helm upgrade --install dx-deployment ... \
     -f lab-base/custom-values.yaml \
     -f lab-ldap/custom-values.ldap.yaml \
     -f lab-bbdd/custom-values.postgres.yaml
kubectl -n dx-demo logs -f dx-deployment-web-engine-0 \
     -c web-engine
# 3-3 · count the tables (~465)
kubectl -n dx-shared exec dx-pg-1 -c postgres -- psql ...
```

Key values

- useExternalDatabase: **true** · dropDatabaseTables: false
- JDBC: **jdbc:postgresql://dx-pg-rw.dx-shared.svc.cluster.local:5432/dx_core?sslmode=verify-full**
- user **dxuser**, one schema per domain
- Result: 465 tables, over TLS

▶ **Live demo:** walk through **lab-bbdd/custom-values.postgres.yaml** and **dx_dbdomain.properties**, follow the transfer log, count the tables.

📖 HCL documentation (CF239): **DX Compose · Database transfer**

AI assistant: LiteLLM and HCL DX IQ

⏱ ~9 min

What we do

- First **LiteLLM**, the LLM gateway of IQ (not HCL software): **OpenRouter** → **Claude**
- Its secrets: the OpenRouter key, the LiteLLM **master key** and the key IQ uses
- IQ = Integrator + MCP Server (chart **hcl-dx-iq 1.0.3**), deployment key with **--set**
- One DX **helm upgrade**: HAProxy routes the IQ chat

Commands · steps 4-1 to 4-5

```
# 4-1 · LiteLLM, with its secrets
kubectl apply -f lab-iq/secrets.yaml
helm upgrade --install litellm oci://ghcr.io/berriai/litellm-helm \
     --version 1.106.5 -n dx-shared -f lab-iq/litellm-values.yaml
# 4-2 · ask Claude through LiteLLM (optional)
curl -s localhost:8000/v1/chat/completions -H 'Authorization: ...'
# 4-3 · install IQ (Integrator and MCP Server)
IQ_DEPLOYMENT_KEY=$(kubectl -n dx-demo get secret iq-deployment-key ...)
helm upgrade --install dx-iq charts/hcl-dx-iq-1.0.3.tgz -n dx-demo \
     -f lab-iq/custom-iq-values.yaml \
     --set configuration.licensing.deploymentKey="$IQ_DEPLOYMENT_KEY"
# 4-4 · enable IQ in DX
helm upgrade --install dx-deployment ... -f lab-base/... \
     ... -f lab-iq/custom-values.iq.yaml
# 4-5 · ask IQ from the terminal (optional)
```

Key values

- LiteLLM: **litellm.dx-shared:4000**
- iq-general-purpose → **Claude Sonnet 4.6**; iq-summary → **Haiku 4.5**
- Deployment key: Secret **iq-deployment-key**, never in a values file
- MCP: **enableWcm, enableDam**
- networking.dxIqService: **dx-iq-integrator**

▶ **Live demo:** install **LiteLLM** and ask “ready?”, then walk through **lab-iq/custom-iq-values.yaml** and the IQ button in the DX toolbar: “Which WCM libraries are there?”

📖 HCL documentation (CF239): **HCL DX IQ · Deploying LiteLLM** · **HCL DX IQ · Installation**

Search with OpenSearch

⏱ ~7 min

What we do

- **Search V2**: OpenSearch + Search Middleware + File Processor (hcl-dx-search 2.37.0)
- Mutual TLS: a lab CA and **admin / node / client** certificates, one command each, and one secret per certificate
- One DX **helm upgrade** enables Search V2

Commands · steps 5-1 to 5-8

```
# 5-1 · a CA for the lab (in lab-search/certs)
openssl req -x509 -newkey rsa:2048 ... -out root-ca.pem
# 5-2 to 5-4 · admin, node and client certificates, one step each
openssl req -new -newkey rsa:2048 ... -out admin.csr
openssl x509 -req -in admin.csr -CA root-ca.pem ... -out admin.pem
# 5-5 · one secret per certificate (also node, client)
kubectl -n dx-demo create secret generic search-admin-cert \
     --from-file=admin.pem --from-file=admin-key.pem \
     --from-file=root-ca.pem
# 5-6 · install hcl-dx-search
helm upgrade --install dx-search charts/hcl-dx-search-2.37.0.tgz \
     -n dx-demo -f lab-search/custom-search-values.yaml
# 5-7 · enable Search V2 in DX
helm upgrade --install dx-deployment ... -f lab-base/... \
     ... -f lab-iq/... -f lab-search/custom-values.search.yaml
```

Key values

- Secrets: **search-admin-cert, search-node-cert, search-client-cert**
- adminDN: **CN=admin,OU=DX Compose Labs,O=EUROPESIP,C=ES**
- Crawlers: **WCM, JCR, portal and DAM**, every 15 min
- dx-deploy-rwo for the OpenSearch data

▶ **Live demo:** walk through **lab-search/custom-search-values.yaml**, then the Search Center and the search API explorer.

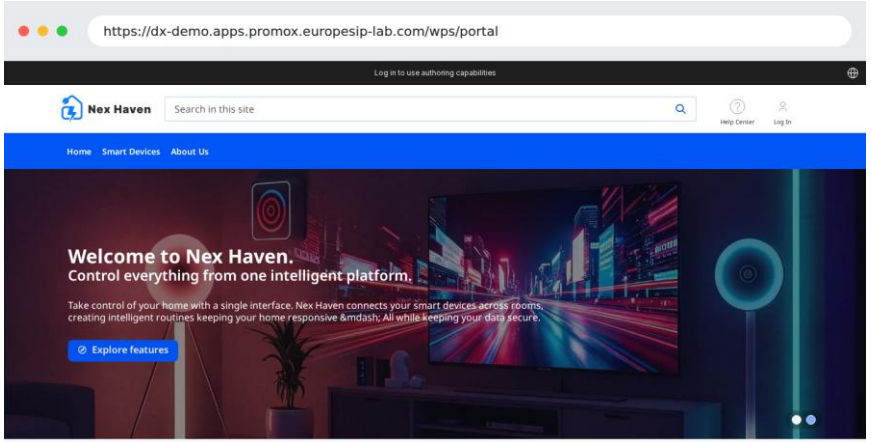
📖 **HCL documentation (CF239):** **DX Compose · Enabling search**

Solution

Deploy DX Compose in Minutes

You can deploy HCL DX Compose, connect to LDAP, transfer to postgreSQL, enable search, IQ and more in no time

1. **Replay this webinar** to review the steps ‘live’ again.
2. **Use the official** documentation details on each step.
3. **Need a hand?** EUROPESIP can help you with any HCL DX installation, in any environment (on premises, cloud or Kubernetes), from a first PoC to a tuned production platform.



Deployed live today: https://dx-demo.apps.promox.europesip-lab.com

europesip

HCLSoftware

Questions?

You can still add any questions to the Q&A panel, if you haven’t already.

We will address any queries that we don’t get to today in our follow-ups.



europesip

References: the showroom and the installer

The showroom

The files, slides and video of this webinar

github.com/europesip/hcl-dx-compose-showroom

DX Installer Wizard · by EuropeSIP

Generates these files, and a full proof of concept, for your own environment

github.com/europesip/hcl-dx-compose-installer

Disclaimer: this lab and webinar cover a **basic, simple, example installation**. Although it is fully functional, there are many more configuration and **tuning** options not covered here. We always recommend **professional customisation services**, especially for sites with high load expectations.

Free HCL DX resources

HCL Digital Experience · free resources

Marketplace, free resources, documentation, HCLSoftware U and more

hclsw.co/dx-free-resources

Start here: the HCL DX home page, the Help Center and its documentation, HCLSoftware U for training, and the HCLSoftware Marketplace with free assets for DX.

Thank you!

Deploy HCL DX Compose in Minutes

github.com/europesip/hcl-dx-compose-showroom
github.com/europesip/hcl-dx-compose-installer
hclsw.co/dx-free-resources